

CUSTOMER REVIEW SENTIMENT ANALYTICS

Big Data Analytics using Python and Machine Learning

Full presentation script, technology explanation and viva preparation

Bharath Rathinasapabathy · 123A8012

SIES Graduate School of Technology, Nerul · Dept. of AI & Data Science · 2025-26

Contents

1. How to use this document	3
1.1 Suggested timing for a 15-minute slot	3
2. Opening script (about 90 seconds).....	5
2.1 The problem statement, mapped to what we built	5
2.2 One-minute elevator pitch	5
2.3 Closing script (30 seconds)	6
3. Technology stack	7
3.1 Overview.....	7
3.2 Why these choices	7
3.3 Architecture in five stages	8
4. How it works	9
4.1 The pipeline, step by step.....	9
4.2 Machine-learning models	9
4.3 The big-data angle	10
4.4 Explainability.....	10
5. Live demo script (about 8 minutes).....	11
6. Results and honesty.....	12
6.1 Measured results	12
6.2 Strongest and weakest classes	12
6.3 Limitations you should state yourselves	12
7. Likely questions and answers	13
8. Setup and troubleshooting.....	16
8.1 Commands.....	16
8.2 If something goes wrong	16
8.3 Project files	16

1. How to use this document

This document contains everything you need to present the project to the examiner: what to say, what to click, how each technology works, and answers to the questions that are likely to be asked. Read it once end-to-end, then rehearse the opening script, the demo script and the question list aloud.

Table 1: Project at a glance

Item	Detail
Project	Customer Review Sentiment Analytics: classify product reviews as positive, negative or neutral.
Course / tools	Big Data Analytics · Python, Pandas, NumPy, Scikit-learn, Jupyter Notebook, Streamlit.
Data	199,426 real Amazon product reviews (English), 1-5 stars, from the Hugging Face hub.
Labels	1-2 stars = negative, 3 = neutral, 4-5 = positive (40 % / 20 % / 40 %).
Model	TF-IDF (unigrams + bigrams, 150,708 features) + Logistic Regression, tuned on a validation set.
Headline result	71.2% accuracy, 0.676 macro-F1, 0.862 ROC-AUC on 39,886 unseen reviews (baseline 40.0%).
Big-data parts	Sparse matrices, scalability study, out-of-core streaming training, Spark mapping.
Run the demo	<code>streamlit run app.py</code> then open http://localhost:8501
Quality	30 automated tests pass; notebook executed top-to-bottom with outputs saved.

Table 2: Parts of this document

Part	Use it for
Section 2	The opening speech (about 90 seconds). Memorise the structure, not the words.
Section 3	Technology stack and why each choice was made; answers 'why did you use X?'.
Section 4	How the pipeline and models work, step by step.
Section 5	Live demo script (about 8 minutes): left column what to say, right column what to do.
Section 6	Results, strengths, weaknesses and the limitations you should state yourself.
Section 7	Thirty-five likely questions with short answers.
Section 8	Setup, commands and troubleshooting.

Delivery tips. Speak slowly. Say the *why* before the *what* ("reviews are too many to read, so we classify them automatically"). Show the live demo early; examiners remember what they see. If you do not know something, say what you would do to find out; never invent numbers. Admit the limitations before the examiner asks; it shows maturity.

1.1 Suggested timing for a 15-minute slot

Table 3: Timing plan

Minutes	Segment	Section
0:00 - 1:30	Opening: problem, data, objective	2
1:30 - 4:00	Method: cleaning, TF-IDF, models, evaluation protocol	4
4:00 - 6:00	Big-data angle: sparse matrices, scalability, streaming	4.3

Minutes	Segment	Section
6:00 - 12:00	Live demo	5
12:00 - 14:00	Results, error analysis, limitations	6
14:00 - 15:00	Closing and future work	2.3

2. Opening script (about 90 seconds)

Say

"Good morning, ma'am. Our project is **Customer Review Sentiment Analytics**, built for Big Data Analytics using Python and machine learning.

"Every online store receives thousands of customer reviews a day. Nobody can read them all, and a star rating alone does not tell you *why* a customer is unhappy. So the problem we solve is: **automatically classify each product review as positive, negative or neutral**, and then learn from the results what customers care about.

"We used **199,426 real Amazon reviews**. Ratings of 1 and 2 stars became negative, 3 stars neutral, and 4 and 5 stars positive. The text is cleaned, converted to numbers with **TF-IDF**, and we compared **six machine-learning models**. The best, Logistic Regression, reaches **71.2% accuracy and 0.68 macro-F1** on 39,886 reviews it never saw during training, compared with 40.0% for simply guessing the most common class.

"This is a big-data problem because text creates huge, sparse data, so we show **sparse matrices, a scalability study and out-of-core streaming training** that learns from the file in chunks without loading it all. Finally, we built a **Streamlit dashboard** where you can type any review, see the prediction with the words that caused it, upload a CSV, and explore the results. I will now explain the method briefly and then show the demo."

2.1 The problem statement, mapped to what we built

Table 4: Mapping the brief to the project

Requirement in the problem statement	Where it is in the project
Classify product reviews	TF-IDF + Logistic Regression, three classes (src/train.py, notebook sections 5-8).
Positive, negative or neutral	Star mapping 1-2 / 3 / 4-5; neutral analysed separately because it is hardest.
Python	All code: preprocess, modeling, train, facts, app, tests.
Pandas, NumPy	Loading, cleaning, grouping, chunked reading; sparse arrays.
Scikit-learn	TfidfVectorizer, HashingVectorizer, 6 classifiers, GridSearchCV, metrics, Pipeline.
Jupyter Notebook	notebooks/Customer_Review_Sentiment_Analysis.ipynb, executed with all outputs.
Streamlit	app.py: six-page interactive dashboard.
Big data	200k reviews, sparse matrices, scaling study, streaming training.

2.2 One-minute elevator pitch

"We built an end-to-end system that reads about 200,000 Amazon reviews and tells you whether each one is positive, neutral or negative with 71% accuracy, explains its decision word by word, and shows that customer service and product quality are what make customers unhappy. It runs on a laptop, scales by streaming, and comes with tests, a notebook and a dashboard."

2.3 Closing script (30 seconds)

"To summarise: we classified customer reviews into three sentiments with a model that is accurate, explainable and cheap to run. We proved our design choices with experiments, we were honest about the neutral class and the noisy labels, and we showed how the approach scales. In future we would add a transformer model and aspect-based sentiment, and port the pipeline to Spark. Thank you. We are happy to take questions."

3. Technology stack

The stack is deliberately small and explainable. Every piece has one job and we can say why we chose it.

3.1 Overview

Table 5: Technology stack

Layer	Technology	What it does in this project	Files
Language	Python 3.13	All data processing, modelling and the app.	everything
Data handling	Pandas, NumPy	Read JSONL (also in chunks), drop duplicates, map labels, summarise.	src/preprocess.py
Sparse maths	SciPy sparse (CSR)	Stores the TF-IDF matrix efficiently.	inside scikit-learn
Features	scikit-learn TfidfVectorizer / HashingVectorizer	Text to numbers; hashing for streaming.	src/modeling.py
Models	scikit-learn: NB, SGD, LinearSVC, LogisticRegression	Six classifiers compared, one selected.	src/train.py
Experiments	Jupyter Notebook	EDA, ablation, grid search, errors, scalability, streaming.	notebooks/*.ipynb
Visualisation	Matplotlib, Seaborn, WordCloud, Plotly	Static figures (notebook) and interactive charts (app).	outputs/figures, app.py
App	Streamlit + custom CSS	Interactive dashboard, six pages.	app.py, assets/style.css
Persistence	Joblib, JSON, CSV	Saved model and metrics so the app starts instantly.	models/, outputs/
Quality	pytest, Streamlit AppTest	30 automated tests.	tests/
Delivery	Git / GitHub	Version control and sharing.	repository

3.2 Why these choices

- **Python and scikit-learn.** The standard for classical machine learning, excellent documentation, and everything runs on a laptop without a GPU.
- **TF-IDF instead of word embeddings.** It is fast, transparent, works well with linear models and keeps every feature interpretable as a word or phrase.
- **Logistic Regression instead of BERT.** Within a fraction of a point of the best classical model, trains in seconds, gives calibrated probabilities and shows exactly which words matter. A transformer needs far more compute and is listed as future work.
- **Bigrams.** They capture phrases like 'not good' and 'stopped working'. The ablation shows about +2 points macro-F1.
- **Sparse matrices.** Text is more than 99.9 % zeros; sparse storage turns an impossible dense matrix (tens of gigabytes) into a few megabytes.
- **Streamlit.** The whole dashboard is plain Python, so the model and the UI share one language and one code base.
- **Joblib model file.** The saved pipeline contains the cleaner, the vectoriser and the classifier, so predictions at serving time are identical to those in training.

3.3 Architecture in five stages

Table 6: The five stages

Stage	Name	What happens
1	Collect	Load ~200k reviews, remove empty/duplicate reviews, map stars to sentiment.
2	Clean	HTML/URL removal, lower-casing, contraction expansion (don't → do not), stop-words removed but negations kept.
3	Vectorise	TF-IDF over unigrams + bigrams into a sparse matrix.
4	Learn & evaluate	Compare six models on validation, tune C, refit, test once.
5	Serve & explain	Streamlit dashboard: single review, batch CSV, performance, insights, word-level explanation.

4. How it works

4.1 The pipeline, step by step

Example review: "I DON'T like it!! Can't recommend, the seller never replied."

Table 7: Pipeline for one review

Step	What happens	Result for the example
1. Clean	Lower-case, expand contractions, strip punctuation and noise.	"i do not like it can not recommend the seller never replied"
2. Remove stop-words	Drop 'the', 'a', 'and'... but keep 'not', 'never', 'very'.	negations survive
3. TF-IDF	Each unigram and bigram gets a weight: frequent in this review but rare overall = high.	e.g. 'not recommend', 'never replied' become features
4. Classify	Logistic Regression multiplies the sparse vector by learned word weights and applies softmax.	negative 95% / neutral 4% / positive 1%
5. Explain	Contribution = TF-IDF value × class coefficient; strongest terms are highlighted.	not recommend, not, never, seller

4.2 Machine-learning models

Selected model: Logistic Regression

- **What it does:** computes a score for each class as a weighted sum of word features, then converts scores to probabilities with softmax.
- **Settings:** C = 0.5 (regularisation: smaller C = simpler model), class_weight = balanced.
- **Why balanced weights:** neutral is only 20 % of the data; weights make a neutral mistake count about twice as much (1.67 vs 0.83).
- **Why selected:** best validation macro-F1 after tuning (0.667, against 0.659 for the best tuned linear SVM), and it also gives calibrated probabilities and interpretable coefficients.

The six models compared (validation set)

Table 8: Validation results

Model	Accuracy	Macro-F1	Train time
Logistic Regression	70.3%	0.663	13.6 s
Linear SVM	72.0%	0.659	4.4 s
Complement Naive Bayes	70.2%	0.619	0.4 s
SGD (modified Huber)	67.0%	0.616	1.9 s
Multinomial Naive Bayes	70.8%	0.597	0.3 s
Baseline (majority class)	40.0%	0.191	0.1 s

Evaluation protocol

- **70 / 10 / 20 stratified split** (train / validation / test); stratified means every part keeps the 40/20/40 class ratio.

- Models are compared and C is tuned **only on validation**. The test set is used **once**, at the end, so the result is an honest estimate.
- TF-IDF vocabulary is fitted on training data only, so no information leaks from validation or test reviews.
- Cross-validated grid search (3 folds) on a subsample re-fits the vectoriser inside each fold.

4.3 The big-data angle

Sparse matrices

Each review uses about 22 of 150,708 possible features, so the matrix is 99.99% zeros. For the test set it takes 7 MB sparse versus 24 GB dense. Training cost is proportional to non-zeros, not to matrix size.

Scalability study

Table 9: More data gives a better model; cost grows roughly linearly

Training reviews	Accuracy	Macro-F1	Total time
3,190	65.6%	0.615	1.0 s
7,977	67.1%	0.631	3.5 s
15,954	68.3%	0.643	5.8 s
39,885	69.6%	0.659	13.9 s
79,770	70.2%	0.666	26.8 s
159,540	71.2%	0.676	71.4 s

Say: "the curve has not flattened, so more reviews would still help; that is the argument for analysing at big-data scale."

Out-of-core streaming training

- Read the file in chunks of 20,000 reviews; never hold the whole dataset in memory.
- **HashingVectorizer** maps each word to one of 2^{20} buckets with a hash function, so no vocabulary has to be stored.
- **SGDClassifier.partial_fit** updates the model chunk by chunk.
- **Pitfall we found:** the raw file is sorted by star rating, so a naive pass gave only 40.0% accuracy (it predicts the last class it saw). We fixed it with an out-of-core shuffle-shard: scatter rows randomly into 8 shard files, then stream them.
- Result after one pass over 159,885 reviews: 71.4% accuracy and 0.654 macro-F1 (in-memory model: 71.2% / 0.676).
- The same design maps to Spark MLlib (HashingTF + IDF + LogisticRegression) for a cluster.

4.4 Explainability

For the predicted class, every term's contribution is its TF-IDF value times its coefficient. The app colours each word by the class it supports and draws a bar chart of the strongest terms, so a user can see, for example, that 'terrible', 'never' and 'stopped working' drove a negative prediction.

5. Live demo script (about 8 minutes)

Left column: what to say. Right column: what to do on screen. **Before you start:** run `streamlit run app.py` and open `http://localhost:8501` (the browser opens automatically). Keep `data/demo_reviews.csv` and `data/sample_reviews.csv` ready.

Table 10: Demo script

What to say	What to do on screen
Overview (45 s). "This is our dashboard. At the top are the headline results: 71.2% accuracy, 0.68 macro-F1, 0.86 AUC on unseen reviews. The five cards explain the pipeline: collect, clean, vectorise, classify, explain."	Stay on Overview . Point at the four metric cards, then the five step cards, then the class-distribution donut (40 / 20 / 40).
Positive example (45 s). "Let me analyse a review. This one is clearly positive." "The model says POSITIVE with about 99 % confidence. The sentiment meter shows +99 on a scale from minus 100 to plus 100."	Click Analyze Review in the sidebar, then the Positive example button. Point to the result card, probability bars and meter.
Explainability (60 s). "The highlighted words show why. 'Love', 'great' and 'easy' are green because they push towards positive. The bar chart ranks the terms, including bigrams like 'worth penny'. This is possible because our model is linear and every word has a weight."	Scroll to Why this prediction? Hover over a highlighted word. Show the contribution bar chart.
Negative and tricky (90 s). "Now a negative review... 'terrible', 'never', 'stopped working' are the evidence. The next one is tricky: 'I can not say I am happy'. A simple model that removes stop-words would delete 'not' and get it wrong. We deliberately keep negations."	Click Negative , wait, then click Tricky . Point out that the prediction is negative and that bigrams such as 'not good' appear in the evidence chart.
Batch analysis (90 s). "A manager has thousands of reviews, not one. Here I upload a CSV, or use our sample of 200 real held-out reviews with true labels." "The mix is shown as a donut, the confidence histogram shows how sure the model is, and because this file has true labels we see 65 % accuracy on this sample. I can filter, and download everything as CSV."	Click Batch Analysis . Upload <code>data/demo_reviews.csv</code> , choose column review , show the charts. Then click Use sample file . Filter by negative and click Download predictions .
Model performance (90 s). "Here is the honest evaluation. The confusion matrix shows positive and negative are recognised well; neutral is the hard class, which I will explain. ROC curves show strong ranking." "On the comparison tab, all six models, and Logistic Regression wins with the best balance. The scalability tab shows accuracy rising with data and our streaming experiment."	Click Model Performance . Toggle the confusion-matrix percentage switch. Click the tabs Model comparison , Scalability , Experiments in turn.
Insights (60 s). "Finally, business insight. Negative reviews are 40 % overall, but among reviews mentioning customer service the figure is about 70 %. Customer service, quality and shipping are the biggest drivers of unhappiness."	Click Data Insights . Scroll to the 'What do unhappy customers talk about?' chart and the phrase charts.
Wrap-up (15 s). "Everything you saw runs from the saved model, with no internet needed."	Optionally open the notebook in VS Code and scroll to the ablation and learning-curve cells.

If the demo breaks: open `docs/screenshots/` (screenshots of every page) or the executed notebook, which contains all results and charts. Never apologise for long; move on.

6. Results and honesty

6.1 Measured results

Table 11: Final results

Measure	Value
Test accuracy	71.18% (majority-class baseline 40.0%)
Macro-F1 / Weighted-F1	0.6757 / 0.7202
ROC-AUC (macro one-vs-rest)	0.8618
F1 per class (negative / neutral / positive)	0.760 / 0.453 / 0.815
Recall per class (negative / neutral / positive)	74% / 51% / 79%
Share of errors involving neutral	85.6%
Single-review latency (median)	4.4 ms
Batch throughput	7,323 reviews per second
Whole training run	165 s
Automated tests	30 passed, 0 failed

6.2 Strongest and weakest classes

Best: **positive** (F1 0.81) and **negative** (F1 0.76). Weakest: **neutral** (F1 0.45, precision 0.41). A 3-star review mixes praise and complaints, so it sits between the other two classes and even human readers would disagree; most errors are positive or negative reviews drawn into the middle class.

6.3 Limitations you should state yourselves

- **Noisy labels.** Star ratings are a proxy for sentiment; some 5-star reviews contain complaints and some 1-star reviews are mis-clicks.
- **Neutral ambiguity.** The neutral class is inherently ambiguous, which caps accuracy for any model.
- **Bag-of-words.** TF-IDF with bigrams cannot understand sarcasm or long-range context; a transformer would help.
- **One domain, one language.** English Amazon reviews only.
- **Keyword themes.** The complaint-theme analysis uses keyword matching, which is approximate.
- **Single machine.** Scalability and streaming were demonstrated on a laptop, not on a real cluster.

7. Likely questions and answers

Keep answers short. If you do not know something, say what you would do to find out; do not invent numbers.

Q1. What is sentiment analysis?

Automatically deciding whether a piece of text expresses a positive, negative or neutral opinion. Here it is applied to product reviews.

Q2. Why is this a big-data problem?

Volume: 199,426 reviews and a matrix with 150,708 columns. Variety: noisy free text. Velocity: new reviews arrive constantly. Veracity: noisy labels. Value: business insight. We also show sparse storage, a scaling study and streaming training.

Q3. Where does the data come from?

Amazon product reviews, English subset of the Multilingual Amazon Reviews Corpus, downloaded from the Hugging Face hub (SetFit/amazon_reviews_multi_en). It is real data, not generated.

Q4. How did you create the three classes?

From the star rating: 1-2 negative, 3 neutral, 4-5 positive. It is the standard way to get labels without manual annotation, but it makes labels slightly noisy.

Q5. What is TF-IDF?

Term frequency times inverse document frequency. A word scores high if it appears often in this review but in few reviews overall, so distinctive words like 'defective' get more weight than common words like 'the'.

Q6. Why bigrams?

They capture short phrases such as 'not good' or 'stopped working' whose meaning differs from the single words. The ablation shows about two points of macro-F1 gained over unigrams.

Q7. Why did you keep negation words?

Standard stop-word lists remove 'not', which turns 'not good' into 'good'. We removed only useless words; the ablation shows that using the standard list loses about 2.6 points of macro-F1.

Q8. Why Logistic Regression and not deep learning or BERT?

It trains in seconds on a CPU, gives calibrated probabilities, is interpretable, and it was the best of the classical models we tried. A transformer would likely score higher but needs far more compute; it is our main future-work item.

Q9. How does Logistic Regression work for three classes?

It computes a linear score per class from the word weights and converts the three scores into probabilities with the softmax function. The largest probability is the prediction.

Q10. What is class imbalance and how did you handle it?

Neutral is only 20 % of the data, so a model could ignore it. We used balanced class weights (neutral counts about twice as much) and judged the model by macro-F1, which treats classes equally.

Q11. Why macro-F1 instead of accuracy?

Accuracy can look fine while a small class is ignored. Macro-F1 averages the F1 of all classes. Our accuracy is 71.2% but macro-F1 is 0.68, which honestly reflects the weak neutral class.

Q12. Is 71 % accuracy good?

For three-class sentiment from noisy star labels, yes: the baseline is 40.0%. We tried four other ideas (5-class training merged to 3 classes, averaging models, a neutral-class threshold, character n-grams) and none

gained more than 0.7 point, so we are near the limit for bag-of-words models. Positive and negative are recognised with F1 of 0.76 to 0.81.

Q13. Why is the neutral class so hard?

A 3-star review mixes praise and complaints, so it overlaps both other classes. Its AUC is 0.76 versus about 0.90 for the others, and 86 % of all errors involve it.

Q14. What are train, validation and test sets?

Train fits the model; validation is used to compare models and tune settings; test is touched once at the end for an unbiased result. We used a stratified 70/10/20 split.

Q15. How did you avoid data leakage?

The TF-IDF vocabulary is fitted on training data only; the grid search re-fits the vectoriser inside each fold; the test set is used once; test reviews are excluded from the streaming run.

Q16. Did the model overfit?

No sign of it: validation macro-F1 (0.667 for the tuned model before refit) and test macro-F1 (0.676) are very close, and larger C values gave worse validation scores, so we chose a stronger regularisation (C = 0.5).

Q17. What is the hyper-parameter C?

The inverse of regularisation strength. Small C keeps weights small and the model simple; large C fits the training data more tightly and risks over-fitting.

Q18. What is cross-validation / grid search?

We split the data into three folds, train on two and test on one, rotate, and average. Grid search tries every combination of n-gram range, min_df and C and keeps the best average macro-F1 (best: unigrams+bigrams, min_df 5, C 0.5).

Q19. Naive Bayes versus Logistic Regression?

Naive Bayes assumes words are independent and just counts; it trains in under a second but is less accurate. Logistic Regression learns weights jointly, so it handles correlated words better.

Q20. Why not an SVM?

The linear SVM was a close second (validation macro-F1 0.659 against 0.667) but gives no probabilities. We chose the model that scored best and explains itself.

Q21. What is a sparse matrix and why use one?

A matrix that stores only non-zero values. Our TF-IDF matrix is 99.99% zeros; sparse it takes 7 MB for the test set, dense it would take 24 GB.

Q22. What is the hashing trick?

Instead of storing a vocabulary, each word is mapped to a bucket number by a hash function. It needs no vocabulary in memory, so it works for streaming and huge vocabularies, at the cost of rare collisions.

Q23. What is out-of-core learning?

Training on data that does not fit in memory by reading it in chunks and updating the model with partial_fit after each chunk. We streamed the file in chunks of 20,000 reviews.

Q24. Why did the streaming model first fail?

Online learners need randomly ordered data, but our raw file is sorted by star rating, so the model only remembered the last classes and scored 40.0%. We confirmed it by printing the label of every 20,000-row block, then fixed it with an out-of-core shuffle into shards, which gave 71.4%. It is a good example of checking assumptions about data order.

Q25. What did the scalability study show?

Accuracy rose from 65.6% with 3,190 reviews to 71.2% with 159,540, while time grew roughly linearly. The curve had not flattened, so more data would still help.

Q26. How would you scale this to millions of reviews?

Use Spark MLlib: Tokenizer, HashingTF, IDF and LogisticRegression in a Pipeline over partitioned Parquet files; the structure is the same as our scikit-learn pipeline.

Q27. How does the explanation work?

For the predicted class, each term's contribution is its TF-IDF value times its coefficient. The app colours words by the class they support and plots the biggest contributions.

Q28. What is ROC-AUC?

The probability that the model ranks a random positive example above a random negative one, averaged one-vs-rest over the classes. Ours is 0.86; 0.5 is chance and 1.0 is perfect.

Q29. What did the error analysis show?

86% of errors involve neutral; short reviews are hardest; the most confident mistakes are often label noise (glowing text with a low rating) or sarcasm.

Q30. Can it handle sarcasm or other languages?

No. A bag-of-words model cannot detect sarcasm, and it was trained on English only. Transformers and multilingual data are future work.

Q31. What business insight did you get?

Among reviews that mention customer service or refunds, 70% are negative, far above the 40 % average; product quality, shipping and durability follow. Ease of use is the happiest theme.

Q32. How did you test the project?

30 automated pytest tests: cleaning rules, label mapping, loader, model sanity (obvious reviews, negation, valid probabilities, unseen text) and every dashboard page via Streamlit's AppTest.

Q33. What was the biggest challenge?

Getting honest results: tuning only on validation, keeping negations, handling the ambiguous neutral class, and debugging the unstable streaming learner.

Q34. What would you improve next?

Fine-tune DistilBERT, add aspect-based sentiment (price, quality, delivery), use human-labelled neutral data, port to Spark, and add drift monitoring.

Q35. What did you personally learn?

That a good analytics project is more than a model: it needs a sound evaluation protocol, honest error analysis, explainability and a way for people to actually use the result.

8. Setup and troubleshooting

8.1 Commands

Table 12: Commands

Task	Command
Install packages	<code>pip install -r requirements.txt</code>
Download the dataset (47 MB)	<code>python -m src.download_data</code>
Retrain everything (about 3 min)	<code>python -m src.train</code>
Run tests	<code>python -m pytest tests -q</code>
Start the dashboard	<code>streamlit run app.py</code>
Open the notebook	<code>jupyter lab notebooks/Customer_Review_Sentiment_Analysis.ipynb</code>

8.2 If something goes wrong

Table 13: Troubleshooting

Problem	Fix
'Model not found' in the app	Run <code>python -m src.train</code> once; it creates <code>models/sentiment_model.joblib</code> .
ModuleNotFoundError: plotly / streamlit	Run <code>pip install -r requirements.txt</code> .
Port 8501 already in use	<code>streamlit run app.py --server.port 8502</code> .
Notebook cannot find <code>src</code>	Open it from the project folder; the first cell adds the parent folder to the path.
Raw dataset missing (notebook or training)	Run <code>python -m src.download_data</code> (needs internet once).
Charts look unstyled	Make sure <code>assets/style.css</code> exists and refresh the page (Ctrl+F5).
Scalability tab empty	Run the notebook once to export <code>outputs/scalability.csv</code> (included in the repository).

8.3 Project files

Table 14: Where things are

File	Purpose
<code>app.py</code> , <code>assets/style.css</code>	Streamlit dashboard and styling.
<code>src/preprocess.py</code>	Loading, labels, cleaning.
<code>src/modeling.py</code>	TF-IDF factory and scoring.
<code>src/train.py</code>	Compare, tune, evaluate, save.
<code>src/facts.py</code>	Latency, throughput, memory, error breakdown.
<code>notebooks/...ipynb</code>	Full analysis with outputs.
<code>tests/</code>	Automated tests.
<code>docs/</code>	Report, this script, diagrams, screenshots.